

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	P.Guru
Name	192521010

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 10

Total Marks: 25

Code of Conduct

I P.Guru /192521010 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Guru

Assessment Tasks

Mock Interview

Answer the following interview-oriented questions clearly and concisely. Support your responses with architecture-level reasoning wherever appropriate.

1. Explain the difference between HDFS and traditional file systems.
2. Why is replication important in distributed storage systems?
3. How does MapReduce divide work between map and reduce phases?
4. What role does YARN play in large-scale Hadoop processing?
5. Differentiate NAS and SAN in an enterprise data environment.
6. What is the purpose of ETL in a big data pipeline?
7. How does Apache NiFi help in data integration workflows?
8. What are the key failure points in a Hadoop cluster and how would you handle them?
9. How would you design a secure data ingestion pipeline for a large organization?
10. How would you explain a scalable Hadoop-based processing system to a non-technical manager?

Mock Interview Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Knowledge	25%	Shows deep and accurate technical understanding	Good technical understanding	Basic understanding	Weak understanding
Problem Solving	20%	Responds with strong architecture-level reasoning	Reasonable reasoning	Basic reasoning	Weak reasoning
Communication	20%	Answers are confident, precise, and professional	Mostly clear communication	Moderate clarity	Weak communication
Practical Awareness	20%	Connects concepts to real-world implementation well	Some practical relevance	Limited practical relevance	No practical linkage
Confidence and Professionalism	15%	Highly confident and professional	Generally professional	Moderately professional	Low confidence and professionalism

1. Explain the difference between HDFS and traditional file systems.

HDFS (Hadoop Distributed File System) is designed to store very large datasets across multiple computers, whereas a traditional file system such as NTFS or ext4 generally manages files on a single machine or storage system.

The main differences are:

HDFS	Traditional File System
Distributed across multiple nodes	Usually associated with one system
Designed for very large files	Designed for general-purpose files
Provides replication for fault tolerance	Usually depends on underlying storage/RAID
Scales horizontally by adding nodes	Scaling often requires upgrading storage/server
Optimized for high-throughput processing	Optimized for general file access
Works closely with Hadoop processing	Independent of Hadoop

Architecture:

Client → NameNode (metadata) → DataNodes (actual data)

For example, a 1 TB file can be divided into HDFS blocks and distributed across several DataNodes. This allows parallel processing and provides fault tolerance.

2. Why is replication important in distributed storage systems?

Replication means maintaining multiple copies of the same data on different storage nodes. It is important because individual servers, disks, or network components can fail.

For example, with a replication factor of 3:

Block A → DataNode 1

Block A → DataNode 2

Block A → DataNode 3

If DataNode 1 fails, the system can retrieve the block from DataNode 2 or 3.

Replication provides:

- **Fault tolerance**
- **High availability**
- **Data durability**

- **Faster recovery**
- Sometimes improved read performance

In HDFS, replicas are preferably distributed across different racks. This protects against both individual server and rack-level failures.

However, replication increases storage consumption. Therefore, the replication factor must be selected based on the importance of the data and business requirements.

3. How does MapReduce divide work between map and reduce phases?

MapReduce divides a large processing problem into smaller parallel tasks.

Map phase:

The input data is divided into splits. Multiple Mapper tasks process these splits independently and generate intermediate key-value pairs.

Example:

Log record → (HTTP_404, 1)

Shuffle and Sort:

Hadoop groups intermediate values having the same key and sends them to the appropriate Reducer.

Reduce phase:

Reducers aggregate or process those grouped values.

Example:

HTTP_404 → [1,1,1,1,1] → 5

The architecture is:

HDFS → Input Splits → Mappers → Shuffle/Sort → Reducers → HDFS

The major advantage is **parallel processing**. Instead of one machine processing the entire dataset, multiple machines process different portions simultaneously.

4. What role does YARN play in large-scale Hadoop processing?

YARN stands for Yet Another Resource Negotiator. It is the resource-management and job-scheduling layer of Hadoop.

Its main responsibilities are:

- Allocating CPU and memory
- Scheduling applications
- Managing containers
- Monitoring application execution
- Detecting failed tasks and supporting recovery
- Allowing multiple frameworks to share the cluster

The main components are:

ResourceManager → NodeManagers → ApplicationMaster → Containers

The ResourceManager manages overall cluster resources, while NodeManagers manage resources on individual worker nodes.

For example, if Spark, MapReduce, and machine-learning jobs are submitted simultaneously, YARN decides how much CPU and memory each application receives.

Therefore, YARN separates **resource management from data processing**, allowing different workloads to share the same Hadoop cluster efficiently.

5. Differentiate NAS and SAN in an enterprise data environment.

NAS (Network Attached Storage) provides **file-level storage**, while **SAN (Storage Area Network)** generally provides **block-level storage**.

NAS	SAN
File-level access	Block-level access
Accessed through network file protocols	Appears to servers as block storage
Easier to configure and manage	More complex infrastructure
Suitable for shared files and documents	Suitable for databases and high-performance workloads
Commonly uses Ethernet networks	Often uses dedicated storage networking

NAS example:

Employees access shared folders containing documents and reports.

SAN example:

An enterprise database server accesses high-performance block storage.

At the architecture level:

NAS:

Client → Network → NAS → Files

SAN:

Server → Storage Network → SAN → Blocks

Both are useful, but the choice depends on performance, workload, management, and application requirements.

6. What is the purpose of ETL in a big data pipeline?

ETL stands for Extract, Transform, Load.

Its purpose is to move data from different sources into a target platform while preparing it for analysis.

Extract: Collect data from databases, files, applications, APIs, etc.

Transform: Clean, validate, filter, standardize, join, or aggregate the data.

Load: Store the processed data in systems such as HDFS, data warehouses, or analytical databases.

Architecture:

Multiple Sources → Extract → Transform → Load → Data Lake/Warehouse → Analytics

For example, customer information may come from a database, sales data from CSV files, and website activity from logs. ETL can combine these sources into a consistent analytical dataset.

ETL improves **data quality, consistency, and usability** before analytics or machine learning.

7. How does Apache NiFi help in data integration workflows?

Apache NiFi is a data-flow and integration platform used to move and manage data between different systems.

It can connect:

- Databases
- Files
- APIs
- Applications
- IoT sources
- Hadoop/HDFS
- Cloud storage

A typical architecture is:

Source → NiFi → Validation → Transformation → Routing → HDFS/Database

NiFi provides features such as:

- Data routing
- Data transformation
- Filtering
- Back-pressure
- Monitoring
- Error handling
- Data provenance
- Flow-based processing

For example, NiFi can continuously monitor a directory, detect a new CSV file, validate it, remove invalid records, add metadata, and send it to HDFS.

Its major advantage is that data flows can be designed visually and monitored without writing an entire custom ingestion application.

8. What are the key failure points in a Hadoop cluster and how would you handle them?

A Hadoop cluster contains several possible failure points.

1. DataNode failure

A DataNode may fail because of disk, hardware, or network problems.

Solution: HDFS replication and automatic re-replication.

2. NameNode failure

The NameNode stores HDFS metadata. Its failure can affect access to the filesystem.

Solution: Use an **Active/Standby NameNode architecture** with automatic failover.

3. NodeManager failure

A NodeManager failure can affect applications running on that worker node.

Solution: YARN detects the failed node and reallocates tasks to healthy nodes.

4. Network failure

Network failures can interrupt communication between nodes.

Solution: Redundant network paths, monitoring, and appropriate rack-aware architecture.

5. Disk failure

Individual disks may fail.

Solution: HDFS replication, disk monitoring, and replacement/recovery procedures.

6. Application/task failure

Individual MapReduce or Spark tasks may fail.

Solution: Hadoop/YARN can restart failed tasks on healthy resources.

Overall architecture:

Monitoring → Failure Detection → Failover/Task Restart → Data Recovery → Re-replication

The objective is to ensure that individual component failures do not become complete system failures.

9. How would you design a secure data ingestion pipeline for a large organization?

I would design the pipeline using **secure connections, authentication, authorization, validation, encryption, and auditing.**

A suitable architecture is:

Data Sources → Secure NiFi/ETL Layer → Validation → HDFS/Data Lake → Processing → Analytics

The major security mechanisms would include:

1. Secure data movement:

Use TLS/HTTPS or other secure protocols to encrypt data during transmission.

2. Authentication:

Verify the identity of users and services. In Hadoop environments, Kerberos can be used for strong authentication.

3. Authorization:

Use role-based access control so users can access only the datasets they are permitted to use.

4. Encryption at rest:

Sensitive data stored in the data lake should be protected using appropriate encryption mechanisms.

5. Data validation:

Validate incoming data before allowing it into production storage.

6. Deduplication:

Use unique identifiers, checksums, or keys to avoid duplicate data.

7. Auditing:

Maintain logs showing who accessed or transferred data and when.

8. Failure handling:

Invalid or suspicious records should be routed to a quarantine area instead of being inserted directly into analytical storage.

Thus, security should be applied throughout the pipeline rather than only at the final storage stage.

10. How would you explain a scalable Hadoop-based processing system to a non-technical manager?

I would explain it using a simple analogy:

"Think of Hadoop as a large team of workers sharing a huge warehouse."

Instead of storing and processing all company data on one powerful computer, Hadoop distributes the data across many ordinary machines.

For example, suppose the company has **100 TB of data**.

Instead of one server trying to store and process everything:

100 TB → One Server

Hadoop distributes it:

100 TB → Many Storage Nodes

Then multiple computers can process different parts of the data simultaneously.

The architecture can be explained simply as:

Data Sources → Distributed Storage → Parallel Processing → Business Results

HDFS acts like the **warehouse** that stores large amounts of information safely.

YARN acts like the **manager** that decides which workers should perform which jobs.

MapReduce or Spark acts like the **workforce** that processes the information in parallel.

Analytics and dashboards act like the **business reporting layer** that converts processed data into useful information.

The biggest advantage is **scalability**. If the business grows and data increases from 100 TB to 500 TB, the organization can add more machines instead of replacing the entire system with one extremely powerful server.